

Content-Domain Boundaries and Enforcement — Decomposing B1.18 by Formalizing How Boundaries Are Authored as DNA Rules, How Out-of-Domain Inputs Are Detected and Handled, How Boundary Conflicts Are Treated as First-Class per A1.03, and How Boundary Changes Require Governed Directed Selection per B1.14

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), the second paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026). It does not introduce new axioms. Its contribution is to formalize, as a standalone operational specification, how content-domain boundaries are authored, enforced, violated, and changed within the CKS architecture — and how boundary conflicts are treated as first-class architectural objects requiring governance resolution rather than silent operational handling.

Abstract

The Coordination Knowledge Substrate (CKS) pattern assigns every entity a content-domain: the specified scope of inputs the entity is designed to process. B1.18 names content-domain as a foundational commitment; B2.89 provided the integrating frame; B2.90 formalized specification requirements. This note, B2.91, formalizes the operational layer beneath specification: how content-domain boundaries are authored as DNA rules per B2.25, how out-of-domain inputs are detected and handled through authored boundary rules, how boundary violations are recorded per A2.40 and may trigger directed selection per B1.14, and how overlapping domains and domain gaps are treated as first-class architectural conflicts per A1.03 requiring governance resolution rather than silent resolution. The note identifies the cellular-membrane-selectivity analog as conceptual scaffold, contrasts the architecture against conventional AI systems that process inputs without domain-membership evaluation, and states the limits of what boundary enforcement guarantees.

1. Why content-domain boundaries and enforcement requires standalone formalization

B1.18 establishes the content-domain as an architectural property of every entity in the CKS pattern: each entity has a specified domain of inputs it is designed to process, and that domain is authored at the DNA layer. B2.89 provided the integrating frame for the five-note content-domain decomposition; B2.90 formalized what a content-domain specification must contain to be adequate — schema

constraints, behavioral defaults, scope boundaries. Neither note addressed the operational layer that gives specification its teeth: the boundary itself, how it is enforced during operation, and what happens when inputs violate it or when multiple entities have conflicting domain claims.

This note occupies the third position in the B1.18 decomposition for a specific reason. Boundary specification (B2.90) tells governance and deployment what a content-domain says; boundary enforcement tells the running architecture what it does when the specified boundary is breached. These are distinct architectural claims. The first is a matter of authored content; the second is a matter of authored behavior under operational conditions. The distinction matters because a deployment could have well-specified content-domain boundaries that are never enforced — entities would then process out-of-domain inputs silently, producing outputs outside their designed operating envelope, with no governance record. CKS does not permit that. The enforcement layer is itself authored, governed, and recorded.

The strategic prior-art purpose of this note is to close a specific gap in the defensive-publication chain. Notes B2.89 and B2.90 establish that CKS entities have content-domains with authored specifications. B2.91 formalizes the enforcement layer as architecturally distinct prior art: authored DNA rules for boundary detection and out-of-domain handling, first-class conflict treatment for overlapping and gapped domains, and directed selection as the governance mechanism for boundary change. B2.92 and B2.93 will complete the five-note decomposition by formalizing content-domain at composition scope and verification scope respectively.

2. Boundary specification as authored DNA rules

The content-domain boundary is the line between what an entity is specified to process and what it is not. In CKS, that boundary is not implicit — it is not inferred from patterns in the entity’s action-layer history, derived from statistical properties of prior inputs, or computed at runtime by the entity’s instinct layer. The boundary is authored as DNA-layer content per B2.25, under the rule-authoring commitment of A2.04.

A complete boundary specification contains four authored components.

Inclusion rules. Inclusion rules define what input types fall within the entity’s content-domain. These are schema constraints in the DNA layer: they specify valid input shapes, allowed content categories, permitted source entities or aspects, and any other structural properties that mark an input as in-domain. An input satisfying all applicable inclusion rules is presumptively in-domain.

Exclusion rules. Exclusion rules explicitly mark certain input types as outside the content-domain. Exclusion rules are not merely the negation of inclusion rules; they author a positive claim that specific input types are out-of-scope for this entity. Where inclusion rules specify what the entity handles, exclusion rules specify what the entity explicitly does not handle — a distinction that becomes important when inputs fall in neither category (the gap case, addressed below under boundary conflicts).

Boundary detection logic. Boundary detection is the operational mechanism by which, when an input arrives, the entity’s behavior substrates evaluate whether it satisfies the inclusion rules,

triggers the exclusion rules, or falls into an unanticipated gap. Detection is also authored in the DNA layer per B2.25. It is not a post-hoc inference from output quality; it is a governing check applied before the entity proceeds to process the input.

Out-of-domain handling rules. When boundary detection identifies an out-of-domain input, authored handling rules specify what the entity does. Four options are available; all four may be authored for a single entity with priority ordering among them.

- *Routing*: the entity routes the out-of-domain input to a more appropriate entity, based on authored routing rules that specify which entities hold which content-domains for potential redirect.
- *Escalation*: the entity escalates the input to an aspect-level or governance-level handler for disposition, without attempting to process it itself.
- *Error response*: the entity returns an authored error or decline-to-process response, surfacing the boundary violation to the caller without further action.
- *Recording*: the entity records the out-of-domain event per A2.40 for governance review and potential boundary refinement, irrespective of which of the first three options it also invokes.

Recording per A2.40 is the only option that applies as a default across all three of the others. Out-of-domain encounters are operational events; they belong in the provenance record. The reason is that recorded violation patterns are the primary data source for boundary refinement — governance cannot improve content-domain specifications without knowing how often and in what form the boundary was encountered.

3. Boundary violation treatment

A boundary violation in CKS is more specific than an out-of-domain encounter. An out-of-domain encounter is any instance where an input falls outside the authored inclusion rules. A boundary violation is a case where an entity receives an input that is clearly within the authored exclusion rules — explicitly out-of-domain — yet proceeds to process it, or where the boundary detection mechanism fails to apply.

Boundary violations are handled under a three-part protocol. First, the violation is recorded per A2.40 as an operational event, with provenance metadata capturing the input identity, the entity, the applicable exclusion rule, and the timestamp. Second, if violation patterns accumulate beyond a deployment-defined threshold, governance is notified — the threshold and notification path are themselves authored DNA content. Third, accumulated violation patterns are candidates for directed selection per B1.14: governance may initiate a boundary refinement to correct the exclusion rule, strengthen the detection mechanism, or reallocate inputs to a different entity.

The directed selection that follows from boundary violation patterns is not automatic. Directed selection in CKS is a governance event, not a system-triggered update. The violation record provides the evidence; governance provides the decision authority. The outcome of directed selection is a new DNA version per B2.69 reflecting the corrected or refined boundary specification, with A6.02 retroactivity ensuring the change applies forward without reclassifying prior operations.

4. A1.03 conflict-as-first-class for boundary conflicts

The most architecturally distinctive feature of CKS content-domain boundary enforcement is not the boundary specification itself but the treatment of boundary conflicts: situations where two or more entities have overlapping domains, or where no entity has a domain covering a particular input type.

Overlapping domains. When two entities each have inclusion rules that admit the same input type, their domains overlap. The same input arriving at the system may be claimed by both entities. In a conventional AI architecture, this ambiguity is typically resolved at runtime by routing logic that picks one handler silently, or by whichever entity happens to receive the input. CKS does not permit silent resolution. Per A1.03, an overlapping domain is an architectural conflict — a structural inconsistency in the authored boundary specifications that requires governance resolution. The conflict is registered in the conflict registry, with the overlapping input type, the two or more entities claiming it, and the provenance of the conflicting specifications all recorded. Governance addresses the conflict through directed selection per B1.14 to clarify the boundary: one entity's inclusion rule may be refined, the other's extended, or a routing rule authored to handle the ambiguity with explicit governance intent.

Domain gaps. When an input type falls in no entity's inclusion rules and is not explicitly covered by any exclusion rule, a domain gap exists. The input has nowhere governed to go. In a conventional AI architecture, gaps are typically handled by error propagation, default-to-any-available-entity routing, or silent discard. CKS treats a domain gap as an architectural conflict per A1.03, equal in status to an overlapping domain. The gap is registered in the conflict registry with the same provenance discipline. Governance addresses through directed selection: an existing entity's boundary is expanded, a new entity is created for the gap, or an explicit routing rule is authored to direct the input to an aspect-level handler.

The A1.03 framing is what distinguishes CKS's treatment from both conventional AI architectures and from simpler boundary-enforcement schemes. A1.03 does not merely permit conflicts to be visible; it requires them to be registered and preserved as first-class architectural objects until governance resolves them. Neither overlapping domains nor domain gaps self-resolve under system operation. They remain in the conflict registry, affecting governance visibility, until directed selection produces a new boundary specification that closes them.

This has an operational implication that follows directly: boundary conflict detection in CKS is not a runtime exception handler. It is a composition-time and deployment-time analysis that runs against the authored content-domain specifications of all entities in a deployment, identifies overlaps and gaps structurally, and registers them as conflicts before the system enters operation. A deployment that has unresolved boundary conflicts is a deployment with known first-class architectural issues registered for governance attention — not a deployment that will silently mishandle inputs until a failure surfaces.

5. Boundary changes as directed selection events

Changes to content-domain boundaries — whether expanding, contracting, or clarifying them — are governance events per B1.14. They are not engineering configuration changes made outside the

governance record. The reason is that boundary changes affect which entities process which inputs, which conflict registrations remain valid, and which out-of-domain handling rules apply. A boundary change made outside the governance record would produce a situation where the authored boundary specification and the operational boundary differ, violating the substrate-as-source-of-truth commitment inherited from Paper 1.

Three boundary change types are recognized.

Boundary expansion. Adding new input types to an entity's content-domain. Expansion requires directed selection per B1.14: governance reviews the proposed expansion, evaluates whether any existing entity's domain would overlap, and either authorizes the expansion or identifies conflicts to resolve first.

Boundary contraction. Removing input types from an entity's content-domain. Contraction may create domain gaps if no other entity covers the removed types; contraction therefore requires the same conflict analysis as expansion before the change is authorized.

Boundary clarification. Refining the authored specification of an existing boundary — sharpening inclusion criteria, adding exclusion rules, improving detection logic — without changing the effective scope. Clarification is still a directed selection event because it produces a new DNA version per B2.69 and because the refined specification may resolve or expose latent conflicts not visible under the prior specification.

In all three cases, A6.02 retroactivity applies: the new boundary specification governs future inputs. Prior operations are not reclassified by the change. An entity that processed an input under the prior specification did so correctly under that specification; the change does not convert those operations into violations. This is the architectural property that makes boundary governance tractable at deployment scale: changes accumulate forward in the DNA version history rather than requiring reanalysis of the prior operational record.

As deployments mature and expand, content-domain boundaries evolve. New use cases add input types that were not anticipated at entity creation; operational experience reveals gaps and overlaps not visible in the initial specification; instinct evolution per B1.10 may shift the capabilities available to an entity in ways that make prior boundaries too narrow or too broad. Governance reviews boundary fit as part of periodic deployment oversight, with the conflict registry and A2.40 violation records as the primary inputs. The evolutionary path is directed selection — not autonomous boundary drift, not silent respecification, but governed change producing traceable DNA versions.

6. What makes content-domain boundary enforcement architecturally distinctive

Conventional AI architectures typically lack explicit content-domain boundary enforcement. A language model receives a prompt and generates a response; whether the prompt falls within any designed operating envelope is not evaluated by the architecture — the model processes what it receives. Multi-agent frameworks may route inputs to specialized agents, but the routing logic is typically application-level code that does not author domain membership rules or register domain conflicts as first-class objects. The result is that domain mismatches in conventional AI architectures

are usually invisible at the architectural level: the system produces output, the output may be poor, and the domain mismatch is diagnosed (if at all) through output quality analysis rather than through governance visibility.

Three properties make CKS boundary enforcement architecturally distinct.

First, boundaries are authored, not inferred. The inclusion rules, exclusion rules, and detection logic are DNA-layer content authored under A2.04. They are substrate content subject to the full governance commitments of the CKS architecture: they can be inspected, modified, and overridden at any time; their change history is tracked under A2.40 provenance; and their evolution follows directed selection per B1.14. An architecture where boundaries are inferred from operational data does not carry these properties.

Second, out-of-domain handling is authored, not default. The four handling options (routing, escalation, error response, recording) are authored choices for each entity, not system defaults applied uniformly. This means governance has visibility and control over how boundary violations are handled operationally — a deployment may choose to route quietly, to escalate loudly, or to error with explanation, and the choice is a governance decision not an implementation detail.

Third, boundary conflicts are first-class per A1.03. The overlap-and-gap treatment under A1.03 is what separates CKS boundary enforcement from even relatively sophisticated boundary management in other architectures. Architectures that track domain assignments may detect overlaps but resolve them at routing time, leaving no governance record. Architectures that surface overlaps as configuration warnings do not carry them in the conflict registry as architectural objects requiring governance resolution. CKS's A1.03 commitment makes boundary conflicts persistent, addressable, and traceable — properties that are valuable precisely because domain boundaries in real deployments are imperfect, and the path to improving them requires visibility into where they fail.

7. The biological analog: cellular membrane selectivity

The biological analog that gives content-domain boundaries their conceptual footing is cellular membrane selectivity. Biological cell membranes do not passively admit all molecules from the extracellular environment; they employ specific transport mechanisms — ion channels, carrier proteins, pumps, receptor-mediated endocytosis — that admit specific molecular species and exclude others. The membrane boundary is structural: it is built into the cell's biology rather than computed at runtime from the molecular environment. A cell that loses membrane selectivity does not operate normally on a broader range of substrates; it loses the functional separation that its operations depend on.

CKS content-domain boundaries are the governed architectural analog. The boundary is structural in the authored-DNA sense: it is specified in the DNA layer before the entity operates, not inferred from the input stream after the entity has processed it. The enforcement mechanism is the authored detection and handling rules rather than a physical membrane, but the architectural function is the same: the entity operates on inputs within its designed domain, and inputs outside that domain trigger authored responses rather than unconstrained processing.

The analog is useful as conceptual scaffold rather than as a technical derivation. The architectural substance of CKS boundary enforcement is entirely in the authored-DNA-rules framing and the A1.03 conflict treatment; the biology supplies a familiar picture of why selective boundary enforcement is architecturally productive and not merely restrictive. A cell without membrane selectivity is not a more capable cell; it is a dysfunctional one. An entity without a content-domain boundary is not a more general-purpose entity; it is one whose operational scope is undefined, whose outputs are uninterpretable without domain context, and whose interactions with other entities cannot be compositionally governed.

8. Inherited Paper 1 commitments

Content-domain boundary enforcement inherits six directly load-bearing commitments from Paper 1 through Paper 2.

A1.03 conflict-as-first-class. Boundary conflicts — overlapping domains and domain gaps — are first-class architectural objects. They are registered, preserved, and addressed through governance. They are not silently resolved at runtime. This is the most foundational inherited commitment for boundary conflict treatment.

A2.04 rule authoring. Inclusion rules, exclusion rules, detection logic, and out-of-domain handling rules are authored under the rule-authoring commitment. They are DNA-layer substrate content, not implementation code outside the governance record.

A2.40 provenance. Out-of-domain encounters, boundary violations, and boundary changes are recorded as operational events with the six-field provenance metadata Paper 1 specifies. The provenance record is the primary data source for boundary refinement.

B1.14 directed selection. Boundary changes — expansion, contraction, clarification — are directed selection events. They follow the horizontal evolution mechanism Paper 2 specifies, producing new DNA versions per B2.69. Boundary evolution is governed, not autonomous.

A6.02 retroactivity. Boundary changes apply forward. Prior operations conducted under the prior specification are not reclassified. The retroactivity commitment is what makes boundary governance safe to exercise: governance can change boundaries without incurring an obligation to reanalyze or invalidate the prior operational record.

A1.01 governance. Content-domain boundary specification, enforcement, violation response, conflict registration, and boundary change are all governed activities. Governance holds authority over the boundary at all times under the three-rights commitment of A1.01.

9. Operational implications

Five operational implications follow from the boundary and enforcement architecture formalized here.

Author boundary rules at entity creation. The content-domain specification per B2.90 is authored as DNA content at entity creation. Boundary enforcement rules — inclusion, exclusion, detection, handling — are part of the DNA layer specification per B2.25, not deferred to operational configuration. An entity without authored boundary rules is an entity without an

enforced content-domain.

Violation recording provides data for boundary refinement. The A2.40 recording option for out-of-domain encounters is not merely an audit trail; it is the feedback mechanism for boundary improvement. Governance uses the violation record to identify where boundaries are drawn imprecisely, where inputs are arriving that no entity covers, and where the routing and escalation rules need adjustment.

Conflict detection runs at composition time. The analysis for overlapping domains and domain gaps is composition-time work — governance runs it when entities are assembled into aspects and Selves, before deployment enters operation. A well-governed deployment does not discover boundary conflicts during operation; it enters operation with known conflicts already registered and in the governance queue.

A1.03 conflict registry is populated with boundary conflicts. The conflict registry that Paper 1's A1.03 establishes is not solely for data conflicts and coordination conflicts; it is also populated with boundary conflicts. The same registry, same resolution path, same governance authority apply.

Cross-partner boundary enforcement requires cross-partner authority. In deployments that span organizational boundaries or partner lines, content-domain boundaries may be claimed by entities from different governance perimeters. Cross-partner boundary enforcement per A2.47 authority distribution requires that the authority to resolve overlapping or gapped domains across partner lines be explicitly specified. A boundary conflict between a partner entity and a home entity is not resolvable by either party's governance alone; it requires the cross-partner authority that A2.47 provides.

10. Limits

Four limits bound what content-domain boundary enforcement guarantees.

Enforcement depends on rule quality. Boundary enforcement is authoritative per the authored rules, not per runtime inference about what the entity should process. A poorly authored inclusion rule will admit inputs the entity was not designed to handle; a poorly authored detection mechanism will miss out-of-domain inputs. Enforcement is only as strong as the authored rules that implement it. This is why the violation recording and directed selection feedback loop matters: governance improves rule quality over time precisely because enforcement alone does not guarantee it.

Boundary changes do not retroactively reclassify prior operations. A6.02 applies. An entity that processed an input under the prior boundary specification processed it correctly under that specification, even if the new boundary specification would classify that input as out-of-domain. The governance consequence of a boundary change is prospective, not retrospective.

Runtime inconsistency with authored boundaries requires governance correction. If an entity's operational behavior diverges from its authored content-domain boundaries — processing inputs the exclusion rules mark as out-of-domain, failing to invoke out-of-domain handling rules

— the correction path is governance: diagnosis, directed selection, new DNA version. The architecture does not self-correct runtime inconsistencies with authored specifications; it surfaces them through violation recording so governance can address them.

A1.03 preserves boundary conflicts; it does not auto-resolve them. The conflict registry holds boundary conflicts until governance resolves them. No runtime resolution mechanism closes conflicts that A1.03 has registered. A deployment with unresolved boundary conflicts continues to operate — entities process inputs within their current specifications — but the conflicts remain in the governance queue as unresolved architectural issues.

Content-domain boundaries are not access control. This limit warrants particular emphasis because it defines the scope of this note relative to the broader CKS architecture. Content-domain boundaries specify what inputs an entity is designed to process; they do not specify who is authorized to call the entity, what aspects an entity belongs to, or what cross-entity operations are permitted. Access control — which callers may invoke which entities, what authority is required for which operations — is a separate architectural layer per A2.47 authority distribution. An input may be in-domain for an entity but submitted by an unauthorized caller; the boundary rules address the first question, not the second. Conflating content-domain boundaries with access control would produce misspecified boundaries that try to do two architecturally distinct jobs.

11. Operational test

A CKS deployment instantiates content-domain boundary enforcement for an entity if and only if: (a) the entity's content-domain boundaries are authored as DNA-layer rules per B2.25 under A2.04, specifying inclusion criteria, exclusion criteria, detection logic, and at least one out-of-domain handling option including A2.40 recording; (b) out-of-domain inputs trigger the authored handling rules during operation, and encounters are recorded in the provenance record per A2.40; (c) overlapping domains and domain gaps are identified at composition time and registered as first-class conflicts in the conflict registry per A1.03, where they remain until governance resolves them through directed selection per B1.14; and (d) boundary changes are directed selection events producing new DNA versions per B2.69, with A6.02 retroactivity applying to all changes.

A system that processes inputs without evaluating domain membership, resolves domain overlaps at runtime without governance registration, or changes entity scope without directed selection does not instantiate content-domain boundary enforcement in the CKS sense.

12. Why naming this as standalone matters

Content-domain boundary enforcement could be treated as an implementation detail of content-domain specification: get the specification right, and enforcement follows. The CKS architecture rejects this framing because specification and enforcement are authored separately, evolve separately, and fail separately. A specification authored without corresponding enforcement rules leaves the boundary as documentation rather than operating constraint. An enforcement mechanism that does not surface violations for governance review has no feedback path for improvement. And neither specification nor enforcement alone addresses the boundary conflict problem: overlapping domains and domain gaps are

visible only when the specifications of multiple entities are analyzed together, and resolvable only through governance directed selection.

Formalizing boundary enforcement as standalone prior art — separate from specification (B2.90) and separate from composition (B2.92) — closes a specific patentable territory: the combination of authored DNA-layer boundary rules, first-class conflict registration per A1.03 for overlapping domains and gaps, A2.40 recording as a default out-of-domain handling option, and directed selection per B1.14 as the governance mechanism for boundary change. Each of these components has prior-art precedents in adjacent literatures (schema validation, capability-based access control, feature-flag governance, API lifecycle management), but the combination under the CKS governance architecture — with A1.03 conflict treatment as the architecturally distinctive element — constitutes a specific derivation from Paper 2’s content-domain commitment that this note places in the prior-art record.

B2.91 is the third of five notes decomposing B1.18. B2.89 established the integrating frame; B2.90 formalized what a content-domain specification must contain; B2.91 (this note) formalizes the boundary and enforcement layer. B2.92 will formalize content-domain at composition scope — how boundaries compose across entities when aspects and Selves are assembled. B2.93 will formalize content-domain verification — how governance confirms that authored boundaries match operational behavior. After B2.93, Phase B2 continues with the B1.19 cross-level access decomposition across B2.94–B2.97 and beyond.

Source papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Content-Domain Boundaries and Enforcement — Decomposing B1.18 by Formalizing How Boundaries Are Authored as DNA Rules, How Out-of-Domain Inputs Are Detected and Handled, How Boundary Conflicts Are Treated as First-Class per A1.03, and How Boundary Changes Require Governed Directed Selection per B1.14*. May 12, 2026. ORCID: 0009-0004-8065-3235.